

Asynchronous-many-task systems: Challenges and opportunities - Scaling an AMR astrophysics code on exascale machines using Kokkos and HPX

The International Journal of High Performance Computing Applications
2025, Vol. 0(0) 1–21
© The Author(s) 2025
Article reuse guidelines:
sagepub.com/journals-permissions
DOI: 10.1177/10943420251386503
journals.sagepub.com/home/hpc



Gregor Daiß¹, Patrick Diehl^{2,3} , Jiakun Yan⁴, John K. Holmen⁵, Rahulkumar Gayatri⁶, Christoph Junghans² , Alexander Straub¹ , Jeff R. Hammond⁷, Dominic Marcello³, Miwako Tsuji⁸, Dirk Pflüger¹ and Hartmut Kaiser³

Abstract

Dynamic and adaptive mesh refinement is pivotal in high-resolution, multi-physics, multi-model simulations, necessitating precise physics resolution in localized areas across expansive domains. Today's supercomputers' extreme heterogeneity presents a significant challenge for dynamically adaptive codes, highlighting the importance of achieving performance portability at scale. Our research focuses on astrophysical simulations, particularly stellar mergers, to elucidate early universe dynamics. We present Octo-Tiger, leveraging Kokkos, HPX, and SIMD for portable performance at scale in complex, massively parallel adaptive multi-physics simulations. Octo-Tiger supports diverse processors, accelerators, and network backends. Experiments demonstrate exceptional scalability across several heterogeneous supercomputers including Perlmutter, Frontier, and Fugaku, encompassing major GPU architectures and x86, ARM, and RISC-V CPUs. Parallel efficiency of 47.59% (110,080 cores and 6880 hybrid A100 GPUs) on a full-system run on Perlmutter (26% HPCG peak performance) and 51.37% (using 32,768 cores and 2048 MI250X) on Frontier are achieved.

Keywords

adaptive mesh refinement, asynchronous-many-task systems, Kokkos, HPX, exascale computing, stellar merger, high performance computing

Introduction

Adaptive mesh refinement (AMR) is a crucial component for many high-resolution, multi-physics, and multi-model simulations. In this work, we are focusing on one such simulation code: the astrophysics application Octo-Tiger. Octo-Tiger simulates binary star systems, allowing one to study their interactions and the various phenomena caused by them. In these simulations, large computational domains are needed, for example, to track the mass leaving the stars; see the streamlines in [Figure 1](#). To resolve the quantity of interest, such as the concentration of ^{16}O , some areas of the large computational domain, like the stars (donor and accretor) and accretion column, require a high resolution, which is why AMR is such a crucial component. To provide an example of such an adaptively refined mesh with Octo-Tiger, [Figure 2](#) shows the mesh close to the merger on the equatorial plane. The color shows the portion of the grid assigned to various computational nodes. The workload of the nodes is unbalanced with respect to the domain. This

results in domains of differing sizes being allocated across nodes to achieve load balancing across the mesh. In addition, some nodes have more cells due to the refined mesh; where the smallest cells have a size of 1.4×10^7 cm and the largest cells have a dimension of 2.24×10^8 cm.

This highlights the various scalability challenges we face from the High-Performance-Computing (HPC) perspective

¹University of Stuttgart, Stuttgart, Germany

²Los Alamos National Laboratory, Los Alamos, NM, USA

³Louisiana State University, Baton Rouge, LA, USA

⁴University of Illinois Urbana-Champaign, Champaign, IL, USA

⁵Oak Ridge National Laboratory, Oak Ridge, TN, USA

⁶Lawrence Berkeley National Laboratory, Berkeley, CA, USA

⁷NVIDIA Helsinki Oy, Helsinki, Finland

⁸RIKEN Center for Computational Science, Kobe, Japan

Corresponding author:

Patrick Diehl, Applied Computer Science (CAI-1), Los Alamos National Laboratory, Mail Stop B287, Los Alamos, NM 87545, USA.

Email: diehlpk@lanl.gov

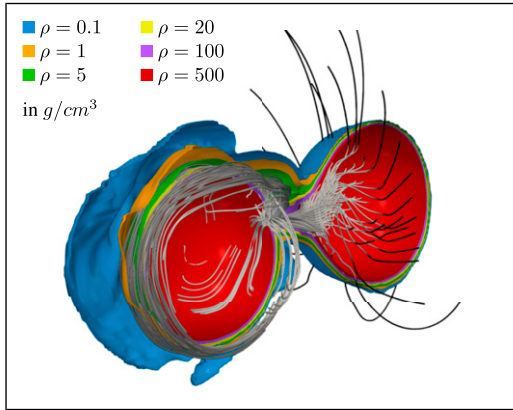


Figure 1. The figure shows an exploded view of the internal structure of a double white dwarf binary star system during the mass transfer phase that precedes the eventual merger of the two stars into a single object. Color is used to indicate density layers (0.1 to 500 g/cm^3). The star to the right of the image (the donor) is transferring mass through a stream striking the upper layers of the more massive white dwarf to the left (the accretor). Several pathlines (gray) show how the stream impacts the accretor, with most of the mass flowing around the accretor while some fraction of the stream spreads in all directions around the point of impact.

when targeting large production simulations on supercomputers with such AMR codes: load-balancing and memory management are fundamental challenges, for example. Another crucial challenge is dealing with the irregular parallelism: AMR implementations usually involve a tree-based data-structure, which poses difficulties when trying to exploit the available parallelism efficiently. Here, we first have to traverse the tree for the work to become available within the system. Especially in large distributed applications, this can lead to starvation of the used supercomputer. Given the diverse set of current supercomputers (including machines with accelerators from NVIDIA, AMD, and Intel) portability is also a concern when developing these large simulation codes.

Given how widespread these concerns are for developing scalable AMR applications, there are existing solutions available to address them. Specifically, for our work, we turn to HPX and Kokkos. HPX, a distributed asynchronous task-based runtime system, can help us to tackle scalability. Expressing the parallelism within Octo-Tiger's tree-structure with the dynamic HPX task-graph and utilizing HPX's distributed features helps with many of the aforementioned challenges, for instance easing the development effort for the distributed memory management, transparent overlapping of computation and communication, and exploiting the parallelism during Octo-Tiger's solver iterations. Kokkos, a framework for developing portable compute kernels, in turn helps us with portability in terms of code and performance. Thus, in theory, combining HPX and Kokkos presents an opportunity to develop distributed and portable HPC applications more easily, helping to achieve

scalability on a wide range of CPU and GPU supercomputers.

However, in practice, we have found that this approach required some additional glue in order for the frameworks to work together seamlessly when developing an AMR application, such as Octo-Tiger. Hence, we needed to develop some integrations and add missing pieces and tools where necessary. In some scenarios, the frameworks themselves did not work well together (such as Kokkos' internal fences (barriers) needlessly blocking HPX worker threads instead of suspending the HPX task).

We also encountered problems regarding missing tools to adapt our use-case to the given hardware. For example, Octo-Tiger was originally developed only for CPUs. When porting Octo-Tiger to Kokkos, the workload per tree-node proved too small to provide sufficient work for an efficient compute kernel (especially on GPUs), causing poor performance due to GPU device starvation. This is a problem shared by many tree-based codes. Moreover, the dynamic nature of the work (i.e., tree-nodes being added, removed, and migrated over the course of the simulation) greatly complicates any static approach for aggregating the workload of different tree-nodes to address this. Over the recent years, we faced and addressed many of these issues and missing pieces in our previous work. Notably, we aimed to do so in a way that makes our solutions usable universally in codes other than Octo-Tiger.

As a part of past work, we improved the interoperability of HPX and Kokkos considerably, allowing us to directly and asynchronously integrate Kokkos operations into the HPX task-graph, eliminating the need to block CPU threads

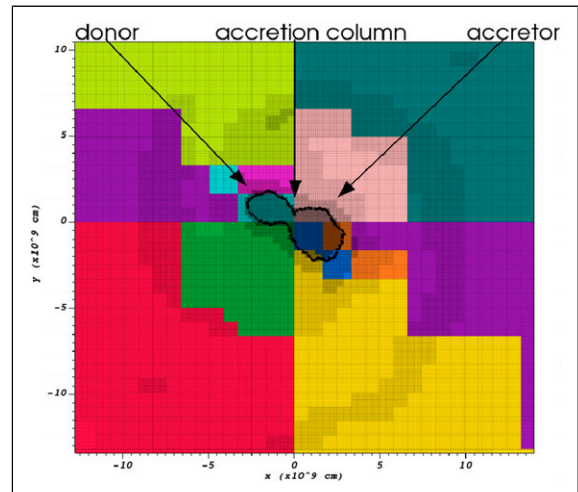


Figure 2. The adaptively refined grid close to the merger for level 11 on an equatorial plane. The contour of the donor star and accretor star are sketched in black. Between the two stars is the accretion column where the mass transfer between the stars occurs. The color shows the portion of the grid assigned to different nodes. The figure is zoomed in on the two stars and does not show the coarser grid further away from the stars.

with blocking fence operations entirely (Daiß et al., 2021). We addressed the device starvation issue by implementing a set of allocators and executors that facilitate dynamic work aggregation, combining individual kernel launches of compatible kernels on-the-fly into a single large compute kernel guided by the current GPU work load (Daiß et al., 2022b). Furthermore, we made use of SIMD types within our Kokkos kernels (allowing for both the Kokkos SIMD types and the `std::simd` types), to improve the efficiency on CPU platforms whilst seamlessly retaining GPU support. Here, we contributed `std::simd`-compatible SIMD types for Scalable Vector Extension (SVE) (Daiß et al., 2022a) and RISC-V Vector Extension (RVV) (Diehl et al., 2024b) to improve platform support, especially for Fugaku (Diehl et al., 2024c). We also added an improved networking backend to HPX (Yan et al., 2023). Each of these previous works considerably improved the usability of HPX and Kokkos for developing scalable and portable AMR applications, thus helping us to realize the opportunities that the combined usage of HPX and Kokkos offers. Over the course of this previous work, we also ported Octo-Tiger’s most important compute kernels to Kokkos, turning it into a GPU-accelerated application. In this work, we build on these achievements, combine all methodologies, enable the seamless exchange of different backends on cutting-edge systems, and show novel scalability results up to full system runs for three of the world’s fastest supercomputers with entirely different hardware. Thus, the work presented here builds directly on our previous work, combining those results in Octo-Tiger to allow one to run it on multiple major supercomputers and examine the scalability and performance of production scenarios at scale on these systems.

Specifically, the contributions of this work include runs on Perlmutter (up to the full system), Frontier (up to 1024 compute nodes), and Fugaku (up to 6144 compute nodes). On Perlmutter, we also include tests using different HPX networking backends. In order to underline the achieved wide portability of Octo-Tiger, we further provide relevant runtime data on several other compute architectures. To us, these runs and the generated runtime data provide useful information regarding where we stand performance-wise with Octo-Tiger and what our next steps should be. To the reader, this provides a real-world use-case of a high-resolution, multi-physics AMR code being scaled to multiple machines using HPX and Kokkos.

Additionally, in this work, we provide a better overview of all of our previous results to make HPX and Kokkos more suitable for applications such as Octo-Tiger. This is to provide the reader with a better insight into what features and framework integrations we had to implement to scale our AMR application to GPU supercomputers in the first place. Crucially, these features and integrations all work independent of Octo-Tiger and are available for other HPC application developers in case they consider using HPX and Kokkos.

As a result, this overview may also prove to be useful for developers of frameworks similar to HPX: it outlines what

challenges need to be overcome for efficient use of a distributed task-based framework with Kokkos, especially when dealing with GPUs and tree-based user codes. The remainder of this work is structured as follows: First, we will mention related work in the next section. Then, as Octo-Tiger is a major part of this work, serving both as motivation and as a benchmark for many of our developments, we will cover its astrophysical use-case and the computational challenges in greater detail afterwards. Next, we will introduce HPX, Kokkos, and Octo-Tiger themselves. Then, we will provide the overview of our previous work that addressed the challenges we found when using HPX and Kokkos in a distributed AMR application. After that, we move to describing the distributed runs we performed on various machines with Octo-Tiger, realizing the opportunities presented by combining HPX and Kokkos, namely scalability and portability. Lastly, given the runtime data collected in the previous section, we discuss our next steps to improve Octo-Tiger and conclude this work.

Related work

Concerning the astrophysics application, the CASTRO (Almgren et al., 2020) application supports adaptive mesh refinement using AMReX (Zhang et al., 2021) with similar astrophysics features. Castro uses the *MPI + X* approach where *X* is OpenMP for CPUs and *X* is either CUDA for NVIDIA GPUs or HIP for AMD GPUs. Octo-Tiger and CASTRO are both astrophysical simulation codes, but they differ in numerical approach and intended use cases. CASTRO is a general-purpose, block-structured AMR code designed for a broad range of compressible flow problems, employing finite volume hydrodynamics and multigrid gravity solvers. Octo-Tiger, in contrast, is specialized for modeling rotating binary star systems. It uses an octree-based AMR mesh with finite volume hydrodynamics that uses the piecewise parabolic method (PPM) to compute the values at all 26 spatial directions on a grid cell. It includes a Self-Consistent Field (SCF) solver capable of generating synchronously rotating binary systems in equilibrium. This is a class of initial conditions that CASTRO cannot natively produce. Octo-Tiger also employs a Fast Multipole Method (FMM) for gravity, including the time derivative of the potential to ensure machine-precision energy conservation in the rotating frame. These differences in both software and scientific focus mean that direct performance comparisons between the two codes are not meaningful, as they are solving fundamentally different problems. Table 1 showcases the supported physics modules. Castro also provides nuclear reaction networks and thermal diffusion, which are not yet available in Octo-Tiger. Other MPI-based solutions are Paramesh (MacNeice et al., 2000), SAMRAI (Wissink et al., 2001), FLASH (Fryxell et al., 2000), p4est (Burstedde et al., 2011), Enzo (O’shea et al., 2005), Chombo (Colella et al., 2012), and deal. II (Bangerth et al., 2007). Focusing on oct-tree

Table 1. Comparison of the physics modules available in Octo-Tiger and Castro.

Method	Castro	Octo-Tiger
Radiation	✓	✓
Gravity	✓	✓
Hydro	✓	✓
Nuclear reaction networks	✓	—
Thermal diffusion	✓	—

based adaptive mesh refinement, tree traversals are needed and depending on the level of refinement d this takes $\mathcal{O}(d)$ global reductions. For large scale simulations and high level of refinements, synchronization and communication overheads can affect the efficiency (Langer et al., 2012). Potential solutions are 1) to use non-blocking MPI calls, 2) overlap communication and computation, and 3) load balancing. The asynchronous launches of HPX tasks easily facilitate non-blocking calls, describing tasks dependencies using HPX’s task graph allows for a natural overlap of communication and computation, and HPX’s support for work stealing of tasks between threads within a node helps with load balancing. A similar approach has been done using Charm++ (Langer et al., 2012).

From the code portability perspective, the following alternatives to Kokkos are available: OCCA, OpenACC, OpenCL, RAJA (Beckingsale et al., 2019), and SYCL. We decided to use Kokkos for the two following reasons: 1) we wanted to use a C++ library since HPX leverages the C++ standard and that would simplify the development of our integrations and 2) AMD, NVIDIA, and Intel GPU support were strong requirements. For the GPUs, we use the respective Kokkos GPU execution spaces. HPX is not the only asynchronous many-task system (AMT) with GPU capabilities in existence. Since this work focuses on distributed runs, we will mention only distributed AMTs. Table 2 shows the GPU support of Chapel (Chamberlain, 2011), Charm++ (Kale and Krishnan, 1993), Legion (Bauer et al., 2012), Uintah (Germain et al., 2000), and PaRSEC (Bosilca et al., 2013). For a detailed survey, we refer to (Thoman et al., 2018). Only Legion and Uintah have Kokkos support. Uintah adopts Kokkos through a Uintah-specific intermediate portability layer used to preserve legacy code and centralize Kokkos calls (Sunderland et al., 2016). Legion allows for Kokkos API calls. To conclude, the novelty of our work is that Legion and Uintah solely use Kokkos’ GPU execution space while we additionally use Kokkos’ HPX backend to launch compute kernels on the CPU.

Overview of the problem

The astrophysical problem

The majority of stars in the Universe are not isolated stars but, rather, members of gravitationally bound systems of stars with two or more components. Binary stars are star

systems with two components that form from the same cloud of gas. The components of binary systems may eventually evolve to the “white dwarf” stage, where nuclear burning has ceased, much of the stellar material has been ejected, and remaining remnants emit only black-body radiation. Known as a double white dwarf (DWD), the components may be driven closer together through emission of gravitational radiation. When the components of a DWD are close enough that the gravity of the “accretor” pulls mass from the “donor,” this is said to be an “interacting” DWD (see Figure 1). Interacting binary systems are commonplace in the Universe. A large fraction of interacting binaries result in the disruption of the donor and subsequent merger of much of its material with the accretor, which makes understanding them crucial to our understanding of a wide variety of potential outcomes for such mergers (e.g., type 1A supernovae, R Coronae Borealis stars).

The R Coronae Borealis stars have strange elemental abundances as they are low in hydrogen but high in carbon content. They are variable stars, fading from and then returning to maximum brightness at intervals of several years. This is thought to be caused by clouds of carbon dust surrounding the star. They are thought to form from the merger of double white dwarfs. Octo-Tiger has been used extensively in the past to study this question. e.g. (Lauer et al., 2019; Marcello et al., 2021; Shiber et al., 2024b; Staff et al., 2012). Octo-Tiger has also been used to investigate whether the star Betelgeuse may be the outcome of a past merger (Chatzopoulos et al., 2020).

The numerical modeling of the short-lived phase of unstable mass transfer in a DWD merger requires a fully three-dimensional, self-consistent treatment of the effects of hydrodynamics and gravity in a rotating frame of reference. There are two basic approaches to the hydrodynamics: Smoothed Particle Hydrodynamics (SPH) and the grid-based finite volume method (FVM). One of the processes crucial to understanding the formation of R Coronae Borealis stars is the creation of an unusually high concentration of ^{16}O in the accretor during merger. SPH codes tend to greatly underestimate this effect, making it difficult to resolve the low mass transfer rates typical of most of the merger processes. FVM codes tend to overestimate it. With FVM, however, it is possible to conduct convergence studies by successively doubling grid resolution to better understand the magnitude of the overestimation.

The computational challenges

The need for application codes to be portable—both in terms of code and performance—is one of the central problems of modern HPC computing. The quickly growing complexity of hardware architectures and software stacks require rapid advancements in development productivity, performance, and software portability. This work relies on standard C++ and widely used portable C++ libraries, such as Kokkos (a

Table 2. Overview of GPU architecture support for various asynchronous many-task systems (AMT). Only HPX, Legion, and Uintah support Kokkos. All of them use Kokkos::Cuda, Kokkos::HIP, Kokkos::SYCL to launch the respective GPU kernels. For the kernel launches on the CPU, Legion and Uintah use the Kokkos::Serial and Kokkos::OpenMP execution spaces. Only HPX provides a Kokkos CPU backend Kokkos::HPX to run Kokkos kernels on HPX threads.

AMT	GPU support			Kokkos		
	NVIDIA	AMD	Intel	Kokkos::OpenMP	Kokkos::HPX	Kokkos::CUDA/HIP/SYCL
Chapel	✓	✓				
Charm++	✓					
Legion	✓	✓	✓	✓		✓
Uintah	✓	✓	✓	✓		✓
PaRSEC	✓					
HPX	✓	✓	✓		✓	✓

C++ Performance Portability Programming Ecosystem (Trott et al., 2022; Edwards et al., 2014)), HPX (a fully C++ standards conforming asynchronous many-task runtime system (Kaiser et al., 2014, 2020, 2023)), and C++ standard SIMD features that have been added to C++26 (The C++ Standards Committee, 2018) to help ensure full code portability of our application. Here, we used an early version of the SIMD features (std::experimental::simd) to allow us to stay compatible with older C++ versions (Octo-Tiger can still be compiled using C++20 compilers). The use of Kokkos, HPX, and SIMD Features guarantees its seamless portability and consistent scaling characteristics across a wide variety of heterogeneous (GPU and non-GPU based) hardware platforms, in particular on Intel, AMD, and NVIDIA products, but also on upcoming ARM64 and RISC-V HPC architectures.

In the problem domain investigated, the amount of computational work per compute kernel is low, requiring adaptive work aggregation on GPU devices to reduce scheduling overheads and starvation effects for the related asynchronous tasks (Daiß et al., 2022b). The nature of highly complex 3D AMR applications like Octo-Tiger (see Figure 2) require exascale computing systems to solve underlying physics problems that require very large amounts of data and pose computational challenges. To achieve high parallel efficiency, these challenges include providing intra- and inter-node load-balancing, pervasive overlapping of computation and communication, tight integration and high utilization of accelerator devices, and runtime-adaptive task placement and scheduling across nodes and diverse execution environments.

Frameworks and application

HPX

HPX is an asynchronous many-task runtime system (AMT). It allows users to express the parallelism within their application by dynamically building a task-graph on-the-fly using HPX futures and continuations. For example, a

function can be launched asynchronously using `hpx::async`, which returns an HPX future. This future can be used to suspend the current HPX task (`get`) or define another task that will be automatically triggered once the original function is done (`then`). Each task can subsequently spawn an arbitrary number of other tasks asynchronously and with little overhead, making parallel work available quickly, which is especially useful for parallelized tree traversals. Of course, there is a lot more functionality available. Particularly, HPX implements all C++23 APIs regarding parallelism and concurrency. For a full list of the functionality available, we refer to HPX’s online documentation¹. The tasks within HPX’s task graph are then processed by a set of HPX worker threads. Thus, HPX can easily handle billions of tasks, which are processed by the worker threads as they become available within the graph.

HPX also contains various key features to enable the development of distributed codes (Diehl et al., 2024a). For example, functions can also be launched asynchronously on other compute-nodes, still getting an HPX future back. There are also communication channels and other distributed features available that tie into the HPX task graph similarly. This allows users to build a distributed task graph. This is made possible by HPX’s Active Global Address Space (AGAS) and various networking backends (parcelports).

Overall, HPX is well suited to address the computational challenges of developing scalable, distributed AMR applications, directly addressing some of the challenges mentioned in Section The computational challenges. With HPX, users can exploit the parallelism within tree-structures more easily by expressing it in a similar graph structure, the HPX task-graph. Through this task graph, which may contain both computation and communication tasks, users can automatically achieve an overlapping of computation and communication. This helps one to scale their codes to numerous compute nodes with a high parallel efficiency. Furthermore, thanks to some of our integrations, we can also incorporate accelerator devices into this task-graph. This helps to keep a high utilization of the accelerator devices by

better overlapping their computations with the CPU tasks and communication.

Use of HPX can also help with portability. HPX's portability is achieved through an interface (API) fully aligned with the latest C++ standard specification. The API decouples the application from the upper layers of the runtime system and hides the low-level details of the underlying architecture. HPX provides general-purpose building blocks (such as C++ standards conforming parallel algorithms), as well as high-level utilities coordinating asynchronous execution, and offers a programming abstraction that facilitates the programming of both shared-memory and distributed-memory systems through a uniform programming interface, scaling up from hand-held devices to HPC clusters. In terms of networking portability, HPX's modular architecture abstracts the concrete networking hardware allowing for Octo-Tiger to be completely independent of it. HPX supports TCP/IP, MPI, LCI (Yan et al., 2023; Snir and Yan, 2023), OpenSHMEM (Poole et al., 2011), and GASNet (Bonachea and Hargrove, 2018) as possible networking conduits. In this work, we focus on two backends (parcels): MPI and LCI.

While HPX does not support developing portable GPU compute kernels, it directly supports integrating existing kernels into the generated asynchronous execution flow by launching those kernels as tasks that are part of the generated HPX task-graph.

Kokkos

Kokkos implements a programming model in C++ for writing performance portable applications targeting all major HPC platforms. For that purpose, it provides abstractions for both parallel execution of code and data management. Kokkos is designed to target complex node architectures with N-level memory hierarchies and multiple types of execution resources. It currently can use CUDA, HIP, SYCL, HPX, OpenMP, and C++ threads as backend programming models (Core Developers, 2024). With Kokkos, users can write their compute kernel once then run it on the correct device by using the Kokkos execution space for the device and the associated Kokkos memory space for data.

Furthermore, Kokkos also contains GPU-compatible SIMD types. On CPU, these can be instantiated to use the appropriate SIMD instructions and registers of the current CPU. On GPU, these can be instantiated to mere scalar values. This allows user to make explicit use of the SIMD resources offered by the CPU, yet retain compatibility with GPUs.

These features help to decouple user code from specific hardware, greatly improving the portability.

Octo-Tiger in a nutshell

Octo-Tiger's domain science use-case. Octo-Tiger is a distributed, full 3D, multi-scale, multi-model, adaptive mesh-

refinement (AMR) astrophysics code based on FVM methods and designed to study stellar mergers as described in Section The Astrophysical Problem. It has been highly optimized for execution on distributed computing systems and can take advantage of the state of the art in GPUs. This allows Octo-Tiger to produce models with the high levels of resolution required to conduct convergence studies and understand the processes related to the increased ^{16}O concentration in DWD mergers that lead to the formation of R Coronae Borealis stars. Octo-Tiger is a real-world application that has proven to solve pressing science domain problems in the field of simulating the merger of two white dwarf stars bound in a binary star system (Diehl et al., 2021; 2024c; Marcello et al., 2021).

The Octo-Tiger code has several features that make it particularly well-suited for modeling interacting binaries. The entire grid structure rotates with the initial orbital frequency of the binary. This reduces the velocity of the material relative to the grid, and therefore reduces the non-physical effects due to numerical viscosity. The code carefully conserves global quantities like linear momentum, angular momentum, and energy, which is important for modeling the initial phase of mass transfer as small violations in conservation of those quantities can significantly effect the simulation results. Octo-Tiger conserves these quantities in the hydrodynamics solver by matching fluxes across jumps in refinement levels. As is the case with most Fast Multipole Method (FMM) solvers, the FMM conserves linear momentum. We also developed a special technique that enables the FMM to simultaneously conserve angular momentum. While the hydrodynamics solver does not conserve angular momentum, conservation in the gravity solver enables Octo-Tiger to conserve energy in the rotating frame; otherwise this would only be possible in a non-rotating frame. The AMR grid allows for the extension of the spatial domain boundaries to orders of magnitude larger than the orbital separation. This allows Octo-Tiger to model the extended low density outflows that occur during the merger process. The AMR grid also enables higher resolution in the regions of interest, making convergence studies more feasible.

Octo-Tiger's components and data-structure. Octo-Tiger models binary star systems as self-gravitating fluids. Hence, it uses two interleaved solvers: It employs a Fast Multipole Method (FMM) gravity solver and a finite volume hydrodynamics solver. A third, experimental solver for radiation is currently in active development. As Octo-Tiger uses a third-order Runge-Kutta integration scheme, each time step involves three iterations of the hydrodynamics solver. As the FMM is modified to conserve angular momentum, each time step further involves actually six (instead of three) iterations of the FMM solver.

The solvers operate on an adaptive octree with the AMR focusing on the atmosphere between the stars, which is

where the mass is being exchanged between stars. Each tree node contains an entire sub-grid with $8 \times 8 \times 8$ (512) cells to improve the computational efficiency.

Each compute kernel typically operates on one such sub-grid (and its ghost layers) at a time. As a result, we deal with a large number of potentially concurrent compute kernels when traversing the tree. However, each of these kernels only processes a comparatively small compute load. In theory, we can further increase the size of the sub-grids at compile-time to compensate, thus increasing the compute load per individual kernel invocation for better efficiency. Yet, this would in turn negatively impact the adaptivity (as we would get a less refined grid given the same amount of overall cells), the scalability (as the sub-grids are the components we distribute onto the compute-nodes) and lastly, it would impact the runtime of the FMM (as it uses the tree-structure to avoid computations by approximations). Hence, both the efficient distributed tree traversals and the efficient handling of the small compute kernels are key to Octo-Tiger’s overall performance. Given the diverse set of available hardware in currently relevant supercomputers (ranging from NVIDIA, AMD and Intel GPU to x86 and ARM CPUs), portability is also a concern. As such, we face similar computational problems within Octo-Tiger as those that are outlined in Section The computational challenges.

Resolving the challenges of scalable AMR applications with HPX and Kokkos

Using HPX in conjunction with Kokkos presents the opportunity to easily develop portable, yet highly scalable, distributed codes, even when using irregular tree-based structures.

For this reason, we turned to both frameworks when developing Octo-Tiger and porting it to GPUs, especially given our small team of core developers with only one developer being available for the refactoring and porting effort. However, we found out that there were some challenges and missing pieces we had to resolve first to make these frameworks work together efficiently and portably (across CPUs and GPUs) within an AMR application such as Octo-Tiger.

- We needed to improve the HPX-Kokkos interoperability, avoiding both the blocking of HPX worker threads with Kokkos fences and avoiding conflicting thread pools for host execution.
- We needed to address the issue of GPU device starvation caused by compute kernels that are too small (in turn, caused by the small workloads per tree-node in AMR applications such as Octo-Tiger). We furthermore needed to resolve the memory allocation overheads caused by temporary (but necessary) GPU buffers.

- We needed additional SIMD types to portably and efficiently target A64Fx and RISC-V CPUs.
- While the MPI parcellport (networking backend) in HPX works well on a range of machines, it is worthwhile to have more alternatives available for the networking that can be interchanged by the user at runtime.

In this section, we outline how we addressed each of those concerns in our previous work. Thus, we introduce the HPX-Kokkos integrations (intended to address the HPX-Kokkos interoperability) (Daiß et al., 2021), specialized allocators and executors (intended to address work starvation and memory overheads for temporary buffers) (Daiß et al., 2022b), additional SIMD types (to enable portable vectorization for A64FX and RISC-V CPUs while maintaining GPU compatibility of the kernels) (Daiß et al., 2022a) and the LCI parcellport (offering another networking choice within HPX) (Yan et al., 2023).

Notably, all of the integrations, backends, and tools mentioned in this Section are not specific to Octo-Tiger and can be used in other HPX applications as well. Yet, in a final part of this section, we outline how these solutions are applied within Octo-Tiger during our efforts to port this application to Kokkos (and GPUs in general).

Combining Kokkos and HPX

We first integrated Kokkos and HPX in (Daiß et al., 2021). Notably, this required an integration both ways: HPX needs to be integrated with Kokkos to enable asynchronous Kokkos calls (kernel launches, deep copies) as part of its own task graph; Kokkos needs to be integrated with HPX to make use of HPX’s thread pool for CPU execution.

The first integration resulted in the HPX-Kokkos interoperability library, which enables the execution of asynchronous Kokkos Kernels (or deep copies) into the HPX task graph. This makes it easy to define continuation tasks that should be triggered once a kernel is done executing, such as doing post-processing on the results or communicating them, easing the development of distributed code. This also eliminates the need for a CPU thread to actively wait on some GPU kernel as long as there is some other work remaining in the queue. This allows the runtime to easily manage thousands of concurrent kernel launches, using hundreds of GPU streams and just a few CPU worker threads on each process (usually one thread per utilized core).

HPX-Kokkos itself is a thin compatibility layer: depending on which Kokkos execution space is used, it uses the appropriate HPX API integrations to get an HPX *Future* for this execution space that represents the event of the encapsulated work being done executing. For instance, when using a Kokkos CUDA execution space, HPX-Kokkos will use the `get_future` functionality of the

underlying HPX-CUDA integration that is directly implemented within HPX using event polling. Similarly, it will use the appropriate integrations for the Kokkos HIP and Kokkos SYCL execution spaces as well (as HPX includes HIP and SYCL integrations which we developed previously (Daiß et al., 2021; Daiß et al., 2023)). HPX-Kokkos merely maps the calls correctly and provides convenience functions such as `deep_copy_async` or `parallel_for_async` that call their Kokkos equivalents (`deep_copy` or `parallel_for`) and immediately obtain a HPX *Future* for those. HPX-Kokkos primarily helps us to integrate the asynchronous Kokkos GPU kernels with HPX, providing benefits to the programmer (it is easy to handle concurrent GPU kernels and define what should happen with their results asynchronously) and improving the runtime (by eliminating active waiting on GPU results in synchronizing barriers or Kokkos fences).

The second integration implements the HPX execution space as part of Kokkos. This Kokkos HPX execution space (included within the Kokkos framework itself) helps with the CPU execution by enabling Kokkos kernels to run directly on the HPX worker threads. This again gives us two advantages: First, it eliminates the need for conflicting thread pools (as we would have when using the OpenMP execution space). Second, it allows us to finely tune how many HPX tasks each Kokkos kernel should be split into (and thus, how many CPU threads are maximally used for their execution). This can be beneficial when dealing with many concurrent kernels, as we do in Octo-Tiger. For

instance, we can use more CPU threads for kernels that work on tree nodes higher up in the tree (where there is less parallel work available) to avoid starving the CPU threads during the tree traversals.

Using Kokkos and HPX together with these integrations allows us to both take full advantage of HPX (allowing us to finely interleave computation and communication tasks while transparently managing asynchronous execution graphs) and Kokkos (allowing us to target different CPU and GPU hardware with a single kernel implementation). Figure 3 shows an example of how a part of such a graph would look like for Octo-Tiger.

GPU-compatible SIMD implementation with SVE support

While using Kokkos itself already allows us to target both CPU and GPU machines with the same kernel implementation, we still need to ensure that the kernel makes use of SIMD vectorization on the CPU. Although relying on auto-vectorization might work in some kernels, the ones in Octo-Tiger contain too many branches, preventing the compiler from automatically vectorizing the code. We can work around this problem by using SIMD masking. However, we still want to do this in a portable way as the code still needs to execute correctly on the GPU.

For this reason, we use C++ SIMD datatypes, specifically we use a common subset of the Kokkos SIMD types and the

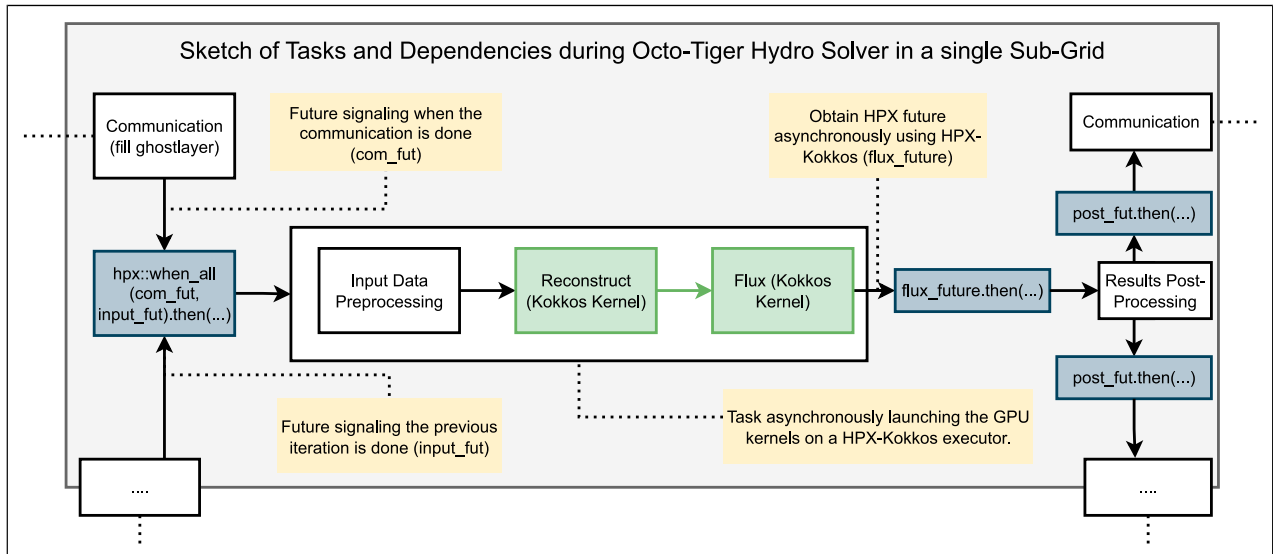


Figure 3. Illustration of a small part of Octo-Tiger’s task-graph, depicting the tasks that are appended to the task-graph for every sub-grid in every iteration of Octo-Tiger’s hydro solver. Tasks (in white) are represented with HPX futures (for instance, named `com_fut`). These futures can be dynamically and asynchronously chained together into a task-graph using HPX functionality (in gray), such as `then` and `when_all`. This allows us to build the task-graph on-the-fly whilst traversing Octo-Tiger’s tree. With the HPX-Kokkos integration, it is possible to integrate asynchronous Kokkos kernels (in green) into this task-graph as well, freeing us from having to use blocking fences to process their results. Overall, this results in a multitude of concurrent CPU tasks, GPU tasks, and communication tasks. This leads to an interleaving of tasks, helping us to better utilize the available hardware resources and achieve scalability.

`std::simd` types (allowing us to decide at compile time, which of those to apply). As our work predates C++26 (and the official introduction of `std::simd`) we used the `std::experimental::simd` implementation within GCC (version 11 and upwards) during our development and testing. However, this is compatible with `std::simd`. Both the Kokkos SIMD types and the `std::experimental::simd` overload all relevant operators and math operations, allowing us to use them just like normal floating point types. During compile time, they get either instantiated based on the relevant SIMD intrinsics (such as AVX512) or as scalar floating point types for kernels that run on the GPU. The types also support SIMD masks for conditional code branching.

We added the SIMD types and masks in all major Octo-Tiger compute kernels and tested the SIMD speedup in (Daiß et al., 2022a) both with the Kokkos SIMD types and with `std::experimental::simd` types from GCC. Furthermore, as there were no `std::simd`-compatible types available for SVE, we implemented these types ourselves and tested them within Octo-Tiger. This allows us to also target the SIMD units in ARM processors (particularly, the A64FX CPUs used in the Supercomputer Fugaku). Eventually, our SVE SIMD type implementation from (Daiß et al., 2022a) has been adapted for inclusion into GCC to benefit the wider HPC community². Similar to the implementation of the SVE types, we also implemented SIMD types for RISC-V and its RVV SIMD instructions (Diehl et al., 2024b). Unlike our SVE types, these have not been yet merged into GCC yet.

Together with the Kokkos HPX execution space, this allows us to have efficient CPU kernel implementations. Usually, we use rather fine-grained kernels where we utilize just 1 to 16 worker threads per kernel instead of the entire CPU. With many of those kernels running concurrently, it allows us to finely interleave computation and communication while at the same time utilizing the SIMD capabilities of the CPU to the best extent possible.

Addressing GPU starvation and memory overheads

While fine-grained compute kernels (as the ones within Octo-Tiger) can be beneficial for adaptivity (more tree levels) and scalability (finer tree structure to distribute onto the compute nodes), it presents major problems for a GPU implementation: On a GPU, we not only have more compute elements than on an average CPU, but we also rely on having enough work items to hide latencies and stalls. At worst, a too small GPU compute kernel with few work items will cause outright device starvation (where we do not even scale to all SMs on an NVIDIA A100). In addition, even when there is enough work for all streaming multiprocessors/compute units, the kernel will still run inefficiently if there are not enough concurrent work items available for latency hiding.

Typically, there are three ways to address this: increase the workload per kernel; increase the number of concurrent

GPU kernels; or fuse kernels and run them as a single, larger GPU kernel. These strategies have their separate trade-offs: for instance, it is possible for Octo-Tiger to increase the workload per kernel by increasing the size of the sub-grid in each tree node. However, this negatively impacts adaptivity (if we aim for a similar grid size) and worsens the FMM performance (as we use the tree-structure to approximate the influence of far-away cells). Hence, it is best to combine it with the other two strategies: running multiple GPU kernels concurrently and employing dynamic GPU kernel fusion.

Handling concurrent GPU kernel launches is straightforward thanks to the integration of Kokkos kernel execution into the HPX task graph via HPX-Kokkos. With this integration, we can easily have 128 concurrent kernels per process, even if the said process only contains a few worker threads to handle launches and communications. However, the overhead of creating a GPU stream (and thus a GPU executor) is significant. To avoid this overhead, we use a pre-allocated pool of GPU executors in each process, combined with a scheduler drawing from it on demand.

To enable the dynamic kernel fusion within HPX, we introduced a special executor in (Daiß et al., 2022b). These executors (enabled by HPX *Futures*) allow users to define code regions where multiple HPX tasks can cooperate by concurrently fusing their individual GPU kernels into one larger kernel (provided it is the same compute kernel operating on different data items). This reduces the overall number of GPU related API calls (fewer kernels and data transfers need to be initiated) and enlarges the work size of the fused kernel (better utilizing the GPU). This is especially valuable on GPUs that are less capable of efficiently running concurrent individual kernels.

We compared these three techniques to avoid GPU starvation and their impact within the Octo-Tiger hydro solver in greater detail in (Daiß et al., 2022b). Here, we also compared the performance of the native CUDA/HIP kernels and their Kokkos counterparts on both NVIDIA A100 and AMD MI100 GPUs.

Lastly, when dealing with many fine-grained compute kernels and associated communication ghost layers, the amount of required temporary GPU buffers can become a problem. Allocating and de-allocating them on-the-fly is too expensive but pre-allocating them for each tree node would significantly increase the memory requirements of Octo-Tiger. We avoid this problem by developing a dynamically growing memory pool for GPU device memory and for pinned memory on the host side. If we need a temporary GPU buffer, we can allocate it with a special allocator that will recycle a (currently unused) GPU buffer from the memory pool. Upon de-allocation, the buffer will simply go back into the pool of unused memory (but stays allocated from the perspective of the system). If there is no unused buffer of sufficient size available upon allocation, a new one will be created. This way, we only allocate as much memory for the temporary and communication buffers as is needed

for maximum concurrency, allowing us to run larger scenarios with fewer compute nodes.

Both the kernel fusion functionality and the memory pool functionality are exposed through specialized executors and allocators which we published in the utility library CPPuddle.

Inter-process communication optimizations

HPX provides users with an Active Global Address Space in which processes can easily register and invoke remote functions and class methods (HPX *actions*) on remote processes or (remote) global objects. This greatly simplifies the programming of Octo-Tiger as we do not need to deal with low-level message transfer. Meanwhile, building Octo-Tiger on top of HPX directly enables us to utilize various advanced HPX network layer techniques to speed up the inter-process communication of Octo-Tiger. This work focuses on two major high-performance network backends (*parcelports*) in HPX: based on MPI and LCI.

The MPI *parcelport* deploys various techniques to enhance communication performance: arguments and return values of remote actions invoked are serialized into short control messages and optional large data messages to minimize memory copies; small messages are aggregated opportunistically to reduce message number; all the messages are sent/received with non-blocking MPI communication primitives to maximize communication overlaps.

Adding on top of the techniques used in the MPI *parcelport*, the LCI *parcelport* (Yan et al., 2023) further employs many communication features that do not exist in current standard MPI, including (a) better communication primitives for active message style communication to eliminate tag matching, ordering, and memory copy overhead; (b) lock-free completion queues for efficient polling of a large number of pending communication operations; (c) lightweight interaction with and replication of low-level communication resources to expose more parallelism from the hardware level. Together, these techniques enhance the HPX communication layer with the ability to more efficiently handle a larger number of (possibly small) concurrent communications in a multi-threaded environment.

Application of our solutions within Octo-Tiger

Our Octo-Tiger application code was written such that it benefits from all aforementioned innovations. It was built from the ground using HPX, thus directly benefits from asynchronous task dependency construction and scheduling as well as intrinsic overlapping of computation and communication embedded in HPX's programming model. Each grid cell of its 3D computational mesh forms a tree-node in the spanned octtree and is represented by one HPX component that can be placed on any compute node. Using HPX's unified syntax for local and remote function calls for

these components, we implemented efficient, distributed tree-traversals with little effort.

While Octo-Tiger was originally developed for CPU-based supercomputers, we gradually ported its computational hotspots to GPUs. This required some major refactoring of the code (requiring changes to some data-structures, like moving to Struct-Of-Arrays for some of them, or moving from an interaction list to a stencil approach for others). Given our small core developer team, this effort was done by a single developer and eventually turned Octo-Tiger into a GPU-accelerated application. However, it is not yet a GPU-resident code: Only the computational-intensive kernels were ported to Kokkos and thus to GPUs, the rest of the application still runs on the CPU. While this already helps us to achieve major GPU speedups with Octo-Tiger (Daiß et al., 2019; 2021; Diehl et al., 2021; Daiß et al., 2022b), it also means that the data is still primarily located on the main memory, not the GPU memory. Thus, there are numerous parts with CPU-GPU communication as the execution flow goes from GPU back to CPU execution, which puts more stress on the CPU-GPU connection and the main memory bandwidth itself, limiting the overall performance achieved by Octo-Tiger. Although there is more work to be done on Octo-Tiger, the existing GPU kernels already speed up the application noticeably in terms of total application runtime. In the context of this work, the frequent CPU-GPU data-transfers and the interleaving of numerous fine-grained CPU and GPU tasks actually provide a good stress test for our integrations and the HPX task-graph processing itself. Regarding Octo-Tiger's GPU kernels themselves, the kernels are written using Kokkos and our HPX-Kokkos integration, ensuring portability. For high efficiency on CPU platforms, we utilize C++ SIMD types within those Kokkos kernels. For instance, we use our SVE types on A64FX systems or scalar types when compiling for GPU platforms. Thanks to the HPX-Kokkos integration, we are able to integrate the asynchronous Kokkos kernels into the HPX task-graph. This enables us to easily handle dozens of concurrent kernel launches per HPX worker thread, as we do not need to keep track of their status ourselves and let the HPX runtime handle this instead, also triggering the subsequent tasks once a Kokkos kernel is done. This automatically gains us the ability to interleave GPU kernels, CPU tasks, CPU-GPU memory transfers, and inter-node communication using the HPX task-graph. In our experience, the resulting automatic interleaving and overlapping of all of these tasks is key for achieving scalability at runtime. Notably, for a GPU-accelerated application, this is done without ever calling Kokkos::fence (or any equivalent synchronize method within CUDA); instead, we are relying on the event polling done by the HPX scheduling system. On GPU platforms, we implement dynamic adaptive kernel fusion (aggregation) techniques to merge individual, concurrent kernel launches into a single GPU kernel to address the GPU starvation

issues. For better memory utilization, we use the recycling allocators as described above.

We make use of the various networking backends offered by HPX that have been improved for this work to reduce networking overheads. We don't have to modify or even recompile the Octo-Tiger code in order to switch networking backends, which makes the performance tuning and comparison seamless.

While all of our contributions were developed in the context of Octo-Tiger, those are completely general and usable in other applications. However, Octo-Tiger serves as a perfect example of the usability of the whole software ecosystem and our contributions to it.

Despite the small Octo-Tiger core developer team, we were able to leverage these innovations to create a portable, highly adaptive simulation software, able to scale to thousands of GPU nodes whilst reducing the runtime per time step below 100 ms even for larger scenarios.

How performance was measured

Tools and utilized scenario

Table 3 details the Octo-Tiger scenarios we use, including the number of cells, the memory usage, and the input file size for all three levels. We use a state obtained from the production runs (Shiber et al., 2024a) close to the merger for all three levels (10, 11, and 12) as a restart file. This allows us to use a very unbalanced mesh with adaptive mesh refinement around the two stars and where the aggregation belt will start, see Figure 2. We used timers in Octo-Tiger to obtain run-time values. However, our timers do not include the I/O time, hence all reported numbers are without I/O.

To make our performance results more comparable with other codes, we include an approximation of the FLOP/s that we obtain during the runs. For this approximation, we obtain the total number of FLOPs for the scenario as follows: Table 3 shows the average floating point operations per time step over ten timesteps measured with the tool *perf* on an Intel Skylake CPU. We compiled Octo-Tiger (using Spack) with no vectorization support and used double precision. These obtained FLOP/s are the basis for all runs on all platforms. The meshes of level 10 and the initial mesh of level 11 fit in the memory of a single node. All other meshes

Table 3. Average floating point operations (FLOP) per timestep measured over ten timesteps using the tool *perf* on an Intel Skylake CPU, Number of cells, the memory usage, and the file size of the input file for all three refinement levels.

Level	FLOP	# Cells	Memory	File size
10	$1.683\,09 \times 10^{12}$	3.8 M	11 GB	548 M
11	$1.748\,06 \times 10^{13}$	40.2 M	113 GB	5.8 GB
12	$1.079\,15 \times 10^{14}$	257.3 M	724 GB	32 GB

required distributed runs and we measured the FLOP on a single node and multiplied it with the number of nodes. This does not accommodate the unbalanced distribution of work. However, as we distribute the workload on as few compute nodes as possible to accommodate for the memory demands, the individual workload per compute node is large enough to make this a good enough approximation for our purposes nonetheless. This approximation is not exact, but provides us with a first estimate of our achieved FLOP/s.

We combined the measured FLOP with *perf* and the timers to calculate the FLOP/s. In the HPC community are two benchmarks to rank supercomputers HPCG and HPL, respectively. HPL ranks the supercomputer's efficiency in solving dense linear equation systems. HPCG uses sparse data structures that have low compute-to-date movement ratios concerning HPL. To compare the obtained FLOP/s, we use the HPCG (Heroux et al., 2013) benchmark results from November 2023. We chose the HPCG benchmark due to its nature of streaming data and executing compute kernels, which aligns more closely with our application. Octo-Tiger adaptively refines the grid and traverses the oct-tree, which includes additional synchronization overhead and increased irregularity not present in the HPCG benchmark.

Systems and environments

The Perlmutter supercomputer hosted at Lawrence Berkeley National Laboratory is a heterogeneous system comprised of AMD EPYC CPUs and NVIDIA A100 GPUs. It has 1536 40 GB GPU nodes, 256 80 GB GPU nodes, and 3000 CPU nodes connected by a 3-level dragonfly topology. We used only the GPU nodes for this study. Each GPU node is comprised of a single AMD CPU with 64 cores and 256 GB of RAM and four A100 GPUs with 40 GB/80 GB of HBM per GPU. Each GPU has 4 HPE Slingshot 11 Network Interface Cards (NICs) and each pair of GPUs is connected via 12 3rd generation NVLINKs with a speed of 25 GB/s per NVLINK.

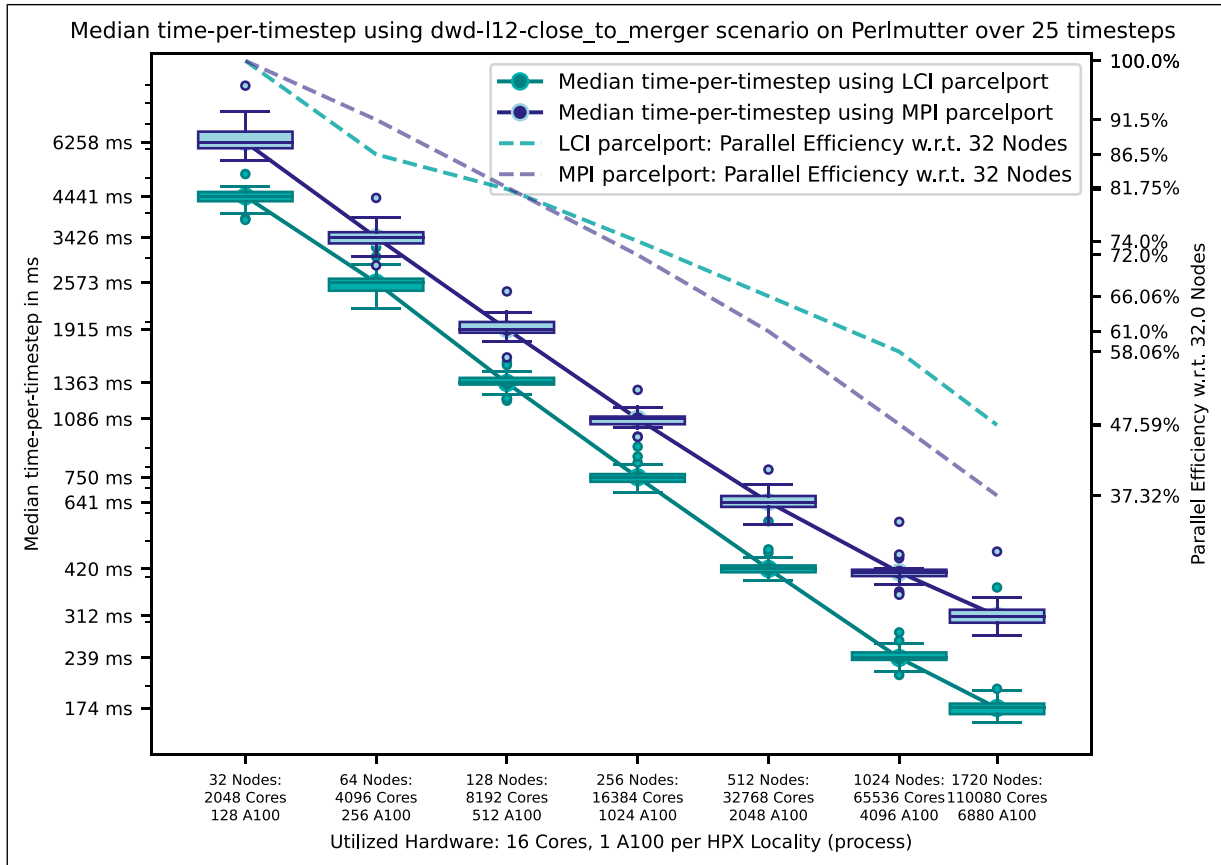
The exascale Frontier supercomputer hosted at Oak Ridge National Laboratory is a heterogeneous system featuring AMD EPYC CPUs and AMD MI250X GPUs. It has 9408 HPE Cray EX235a nodes connected by a dragonfly topology. Each node features one 64-core AMD EPYC 7A53 "Optimized 3rd Gen EPYC" CPU, four AMD MI250X GPUs, each with 2 Graphics Compute Dies (GCDs), 64 GB of high-bandwidth memory (HBM2E) on each GCD, and 512 GB of DDR4 memory. The CPU is connected to each GCD by AMD's Infinity Fabric, which delivers up to 36 + 36GB/s. Each GCD on a node is interconnected by AMD's Infinity Fabric, which delivers up to 50 + 50GB/s for GCDs across GPUs and up to 200 + 200GB/s for GCDs on the same GPU. Each node is connected to the network by 4 Slingshot 11 NICs.

Table 4. Software environments used on the systems. Some systems required patches and we provided the short git hash in *italic*.

System	Compiler	mpi	hpx	Kokkos	hpx-kokkos	GPU SDK	lci
Fugaku	gcc 12.2.0	Fujitsu-mpi head	1.9.1	4.0.01	0.4.0	–	–
Perlmutter	gcc 12.3.0	Cray-mpich 8.1.28	<i>310f449</i>	4.0.01	0.4.0	CUDA 12.2	<i>6dcb6a8</i>
Frontier	rocmcc 5.7.0	Cray-mpich 8.1.28	<i>310f449</i>	4.0.01	0.4.0	HIP 5.7.0	–
Ookami	gcc 12.2.0	Openmpi 4.1.5	<i>40eef48</i>	4.1.00	0.4.0	–	<i>6dcb6a8</i>

The supercomputer Fugaku is a massively parallel computer system with 158,976 nodes based on A64FX (Sato et al., 2020) cores. The A64FX is based on the Armv8.2 architecture with scalable vector extensions (SVE). Each processor has four groups of cores called Core Memory Group (CMG) connected to a ring-bus network. Each CMG contains 12 computation cores and 1 or 0 assistant cores. A computation core has 64KiB L1I and 64KiB L1D caches. The cores in a CMG share an 8MiB L2 cache and 8GiB HMB2 memory. The total size of the memory in a node is 32GiB and the aggregate memory bandwidth is 1024 GB/s. The network of Fugaku is called Tofu Interconnect D (Tofu-D), which is a hybrid 6D mesh/torus network of 40.8 GB/s total injection bandwidth per node (Ajima et al., 2018). Ookami is a small NSF-funded research

cluster at Stony Brook University with 174 nodes containing A64FX CPUs and 32 GB RAM. All nodes are connected with InfiniBand HDR 200. Except for the network architecture, it is similar to the supercomputer Fugaku. Darwin is a research testbed cluster funded by the Computational Systems and Software Environments (CSSE) subprogram of LANL’s ASC program (NNSA/DOE). It is a very heterogeneous cluster with a wide variety of hardware available, including x86, Power PC, and ARM CPU architectures, systems with terabytes of memory, and a variety of GPUs and other accelerators. For this work, we used only the NVIDIA Grace-Hopper partition of the machine. The MILK-V Pioneer is the first workstation-grade development machine for exploring RISC-V. Containing a Sophon SG2042 64-core RISC-V CPU with a 2 GHz clock

**Figure 4.** Median runtime per computational timestep and parallel strong scaling efficiency of the kernel execution per timestep on Perlmutter comparing the use of the MPI and LCI HPX parcellports.

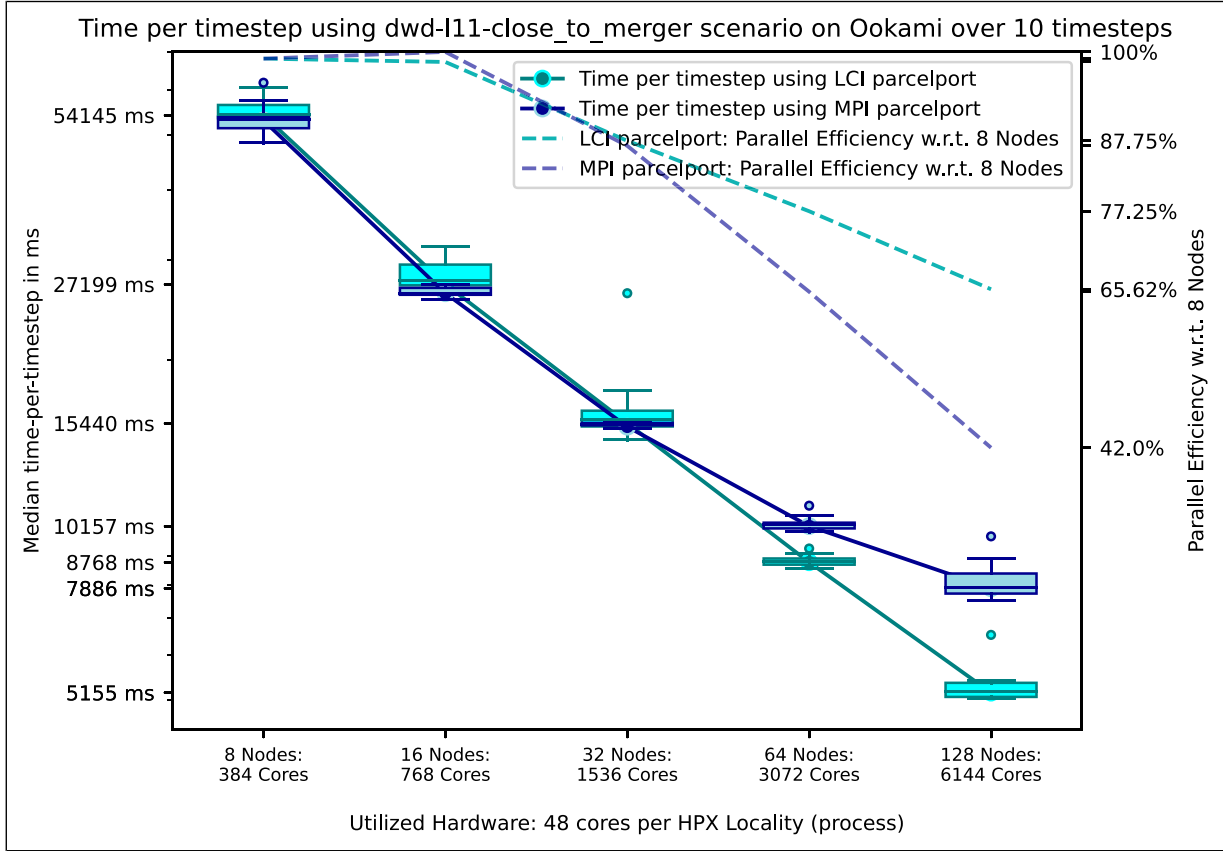


Figure 5. Total Runtime and parallel strong scaling efficiency on Ookami comparing the use of the MPI and LCI HPX parcelports.

frequency, 128 GB DDR 4 memory, and a 1 TB PCIe 3.0 SSD. Table 4 summarizes the used compilers and environments. The data artifact is available on GitHub³.

Performance results

In this section we show the scalability and performance of Octo-Tiger. We will use four platforms: Perlmutter, Frontier, and Fugaku as the major ones; Ookami as an A64FX machine compatible with LCI. First, we focus on the scalability difference between the two relevant HPX networking backends for inter-process communication (LCI and MPI parcelports). Here, we limit ourselves to Perlmutter and Ookami (as we could not test LCI on Fugaku due to its incompatibility with the Tofu-D interconnect nor on Frontier due to time limits). Afterward, we follow up by showing the FLOP/s (based on our approximation of the flops needed by the scenario) and cell-processed per second (to stay comparable to our previous results).

Alternative communication backend

To analyze the effects of using different HPX parcelports at scale, we ran the largest scenario (level 12) on Perlmutter going from 32 nodes up to a full system run. For the full

system run, we achieve a total time of 5.61s using the LCI parcelport and 9.7s when using the MPI parcelport, gaining us a speedup of 1.73x with LCI. The parallel efficiency with respect to the smallest run with 32 nodes is about 42% with the LCI and 35% with the MPI parcelport. Overall, on Perlmutter the LCI parcelport consistently performs better than the (older) MPI parcelport within HPX. For the full system runs, we achieve a speedup of 1.73x using the LCI parcelport over the MPI one. From the perspective of Octo-Tiger core developers, this is especially beneficial since we can reap this LCI speedup without having to modify anything about Octo-Tiger (as the networking backend within HPX is abstracted away).

This Octo-Tiger total time was measured over 25 time steps on Perlmutter (10 time steps on Ookami). As production run scenarios usually take tens of thousands of time steps, we show the scaling in terms of the median runtime per timestep in Figure 4 (with a slightly better parallel efficiency of 47.6% than for the total time). Here, it is worth highlighting that this runtime per time step (including all computations and communication for the multiple, required solver iterations) just takes about 174 ms. Here, we benefit massively from our fine interleaving of computation and communication.

The outliers in Figure 4 are worth mentioning: Here, we can see the effect of the dynamic GPU memory pools that

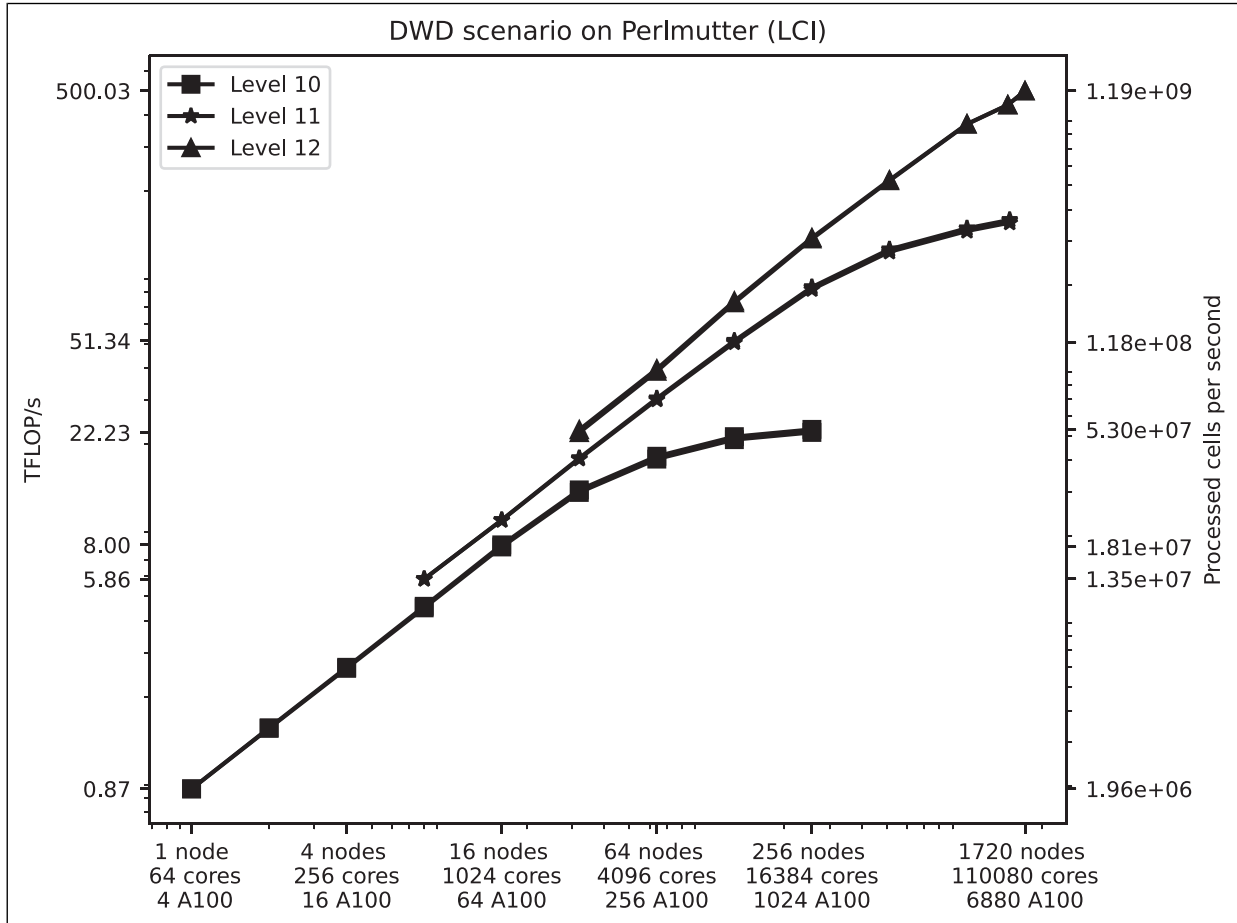


Figure 6. TFLOP/s (left axis) and number of processed cells per second (right axis) on Perlmutter using LCI for three levels of refinement (level 10, 11, and 12).

Octo-Tiger uses. If a GPU buffer is required, the system will try to draw one from these pools and only create one if none is currently available. During the first time step, there are no buffers in the pool yet, hence it is filled over time during this first time step.

We ran the smaller (level 11) scenario with 10 timesteps on the A64FX Oukami machine as well. While the machine is a lot smaller than Perlmutter, it allowed us to test the runtime with both MPI and LCI parcelports on an A64FX platform as well (which we could not do on Fugaku as LCI is not yet compatible with the Tofu-D interconnect). The results can be seen in Figure 5. While the MPI parcelport performs slightly better than its LCI counterpart at smaller node counts, the LCI parcelport achieves significantly better performance at higher scales (starting with 64 nodes). Here, we achieve a LCI speedup of 1.53x when using LCI over MPI with 128 nodes.

Large-scale runs

The previous section demonstrated the scalability, low runtime per time step, and the speedup we can achieve

simply by switching to the LCI parcelport. In this section, we show the total FLOP/s Octo-Tiger can achieve when performing large-scale runs on three major platforms. We use the estimated FLOP/s for each scenario in Table 3. Overall, the simulation is not compute-bound as Octo-Tiger has to deal with adaptive mesh refinement, ghost layer exchanges for each sub-grid, and many smaller auxiliary functions that need to iterate over the data. Furthermore, data-structure conversions within Octo-Tiger allowing the CPU-only parts of the code to interact with the GPU-accelerated code are required to ensure best possible performance on both, the CPUs and GPUs. In general, the workloads of Octo-Tiger are more irregular and fine-grained than those in the HPCG benchmark. It is expected that Octo-Tiger cannot achieve a significant fraction of the total peak performance of the underlying systems.

Figure 6 shows the achieved TFLOP/s (10^{12} FLOP/s) and the number of processed cells per second on Perlmutter. Note that we show the number of processed cells per second to make the runs on Perlmutter comparable with Frontier and supercomputer Fugaku. We show the LCI results here due to the better performance. On Perlmutter, we scale up to

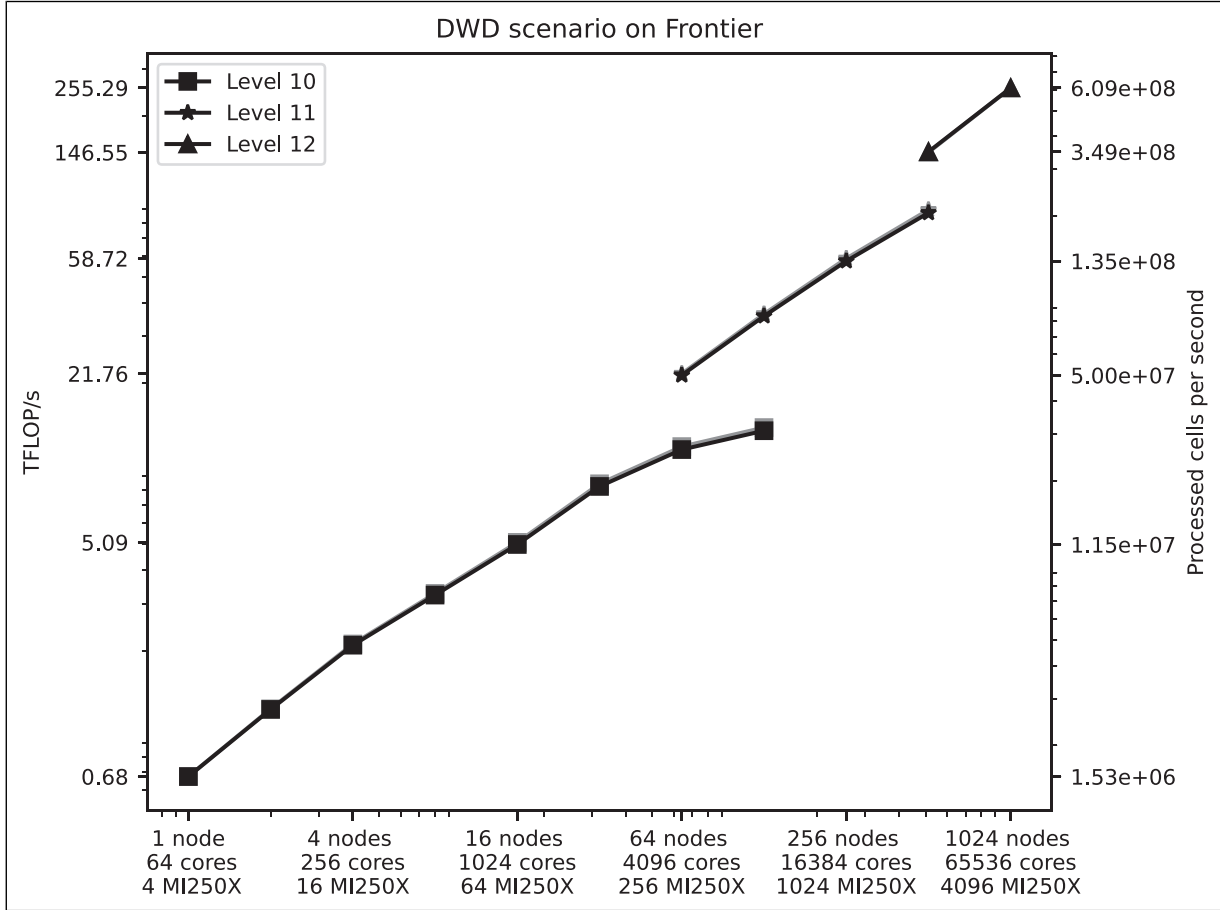


Figure 7. TFLOP/s (left axis) and number of processed cells per second (right axis) on Frontier for an increasing amount of utilized nodes for three levels of refinement (levels 10, 11, and 12).

256 nodes with level 10 having around 3711 cells per GPU per time step. For level 11, we scale up to 1475 nodes with an average of 6814 cells per GPU per time step. Though Perlmutter features 1536 40 GB GPUs, only 1475 were available during our reservation for the full system run. Combining the 40 GB GPUs and the 80 GB GPUs with a higher memory bandwidth allowed for the largest run of 1720 nodes. Here, each GPU had on average 37,398 cells per GPU per time step.

Figure 7 shows the achieved TFLOP/s and the number of processed cells per second on Frontier. Level 10 scales up to 128 nodes with an average of 7422 cells per GPU per time step. Level 11 scaled up to 512 nodes with an average of 19,629 cells per GPU per time step. Level 12 runs finished up to 1024 nodes with an average of 62,817 cells per GPU per time step. On Frontier, for the level 11 scenario, we get a total time 20.89s with 64 nodes. Going to 512 nodes, we get a total time of 5.08s, gaining us a parallel efficiency of 51.38% (regarding the run smallest level 11 run with 64 nodes). While the scalability still looks good (especially for the larger level 12 scenario), we were not able to go beyond 1024 nodes due to the queue waiting times leading up to the submission date.

Figure 8 shows the achieved TFLOP/s and the number of processed cells per second on supercomputer Fugaku for all three levels. We used the SVE backend for our SIMD types. Level 10 scaled from a single node up to 2048 nodes with around 40 cells per core per time step. Level 11 fit in 8 nodes and scaled up to 6000 nodes with around 140 cells per core per time step. Level 12 runs from 1024 nodes up to 6144 nodes with around 872 cells per core per time step. We need to mention here that going from level 11 to level 12 the number of cells and the memory usage increased by 6.4, see Table 3. Note that the largest run was on 294,912 cores. Here, we reached a total time 97.14s with a parallel efficiency 64.5% (with respect to the smallest level 12 run using 1024 nodes). A notable difference is that the Fujitsu MPI used on supercomputer Fugaku only supports `MPI_THREAD_SERIALIZED` and we had to add additional synchronization in our code. However, we could use `MPI_THREAD_MULTIPLE` for the other systems. The additional synchronization overhead shows up for level 12 with many more messages. For a detailed study on the MPI threading level, we refer to (Diehl et al., 2024c). We observe fewer TFLOP/s for level 12 than for level 11 on

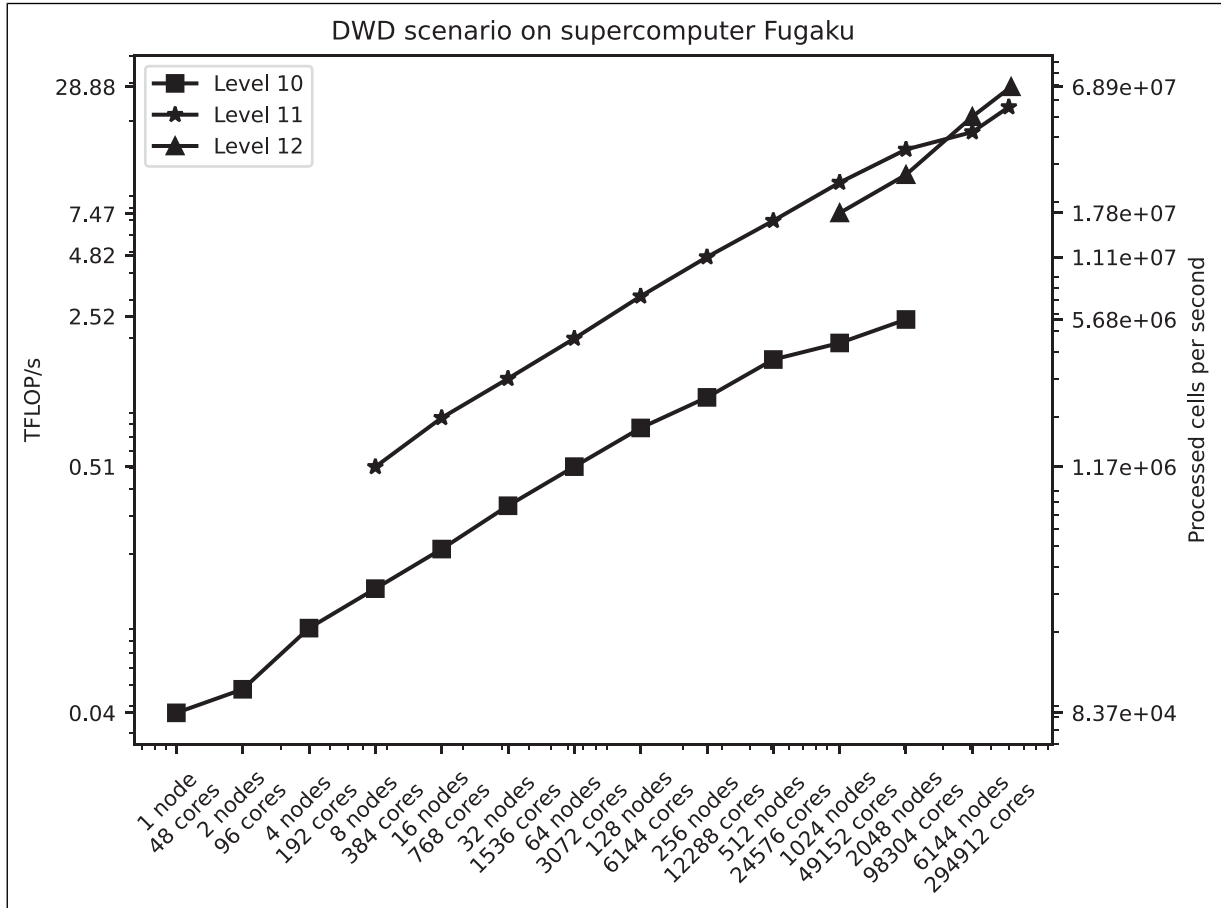


Figure 8. TFLOP/s (left axis) and number of processed cells per second (right axis) on supercomputer Fugaku for three levels of refinement (levels 10, 11, and 12).

A64FX, which we do not observe on Frontier and Perlmutter. For this reason, we have not performed runs on even larger node counts on supercomputer Fugaku.

We are not sure why there is occasional, although rare, superlinear scaling in Figure 8 as this is not cache related (otherwise the superlinear scaling would occur regularly, which it does not). We suspect this was caused by the current network utilization on Fugaku during the time of our runs, but we cannot say for certain.

Table 5. TFLOP/s on the largest node count for three refinement levels and the corresponding percentage of HPCG peak performance in parentheses. Note that HPCG numbers are reported for full system runs which is shown here for level 12 on Perlmutter. For all other runs, we scaled the HPCG full run to the corresponding node count.

Machine/Level	10	11	12
Frontier	13.15 (6.93)	85.88 (22.62)	255.29 (16.81)
Fugaku	2.52 (1.22)	24.10 (3.99)	28.88 (4.67)
Perlmutter (MPI)	12.36 (9.08)	86.79 (6.53)	278.07 (15.20)
Perlmutter (LCI)	20.61 (15.14)	130.73 (9.83)	480.80 (26.28)

Table 5 shows the achieved TFLOP/s for the largest node count on each level of refinement. We compare the obtained TFLOP/s with the peak performance reported by the HPCG benchmark, see Table 6, and report the percentage.

We observed the lowest TFLOP/s on supercomputer Fugaku compared to the two machines with GPU acceleration. The full system run on Perlmutter using mixed GPUs achieved almost twice the number of TFLOP/s (500.03 TFLOP/s) as the run using 1024 Frontier nodes (255.29 TFLOP/s).

Table 7 shows distributed scaling on up to two MILK RISC-V nodes for refinement level 10. We observe an improvement of a factor of around 1.7x going from one node

Table 6. HPCG results (Heroux et al., 2013) from November 2023.

System	HPCG (PFLOP/s)	Frac of peak %
Supercomputer Fugaku	16	3.0
Perlmutter	1.9	2.4
Frontier	14	1.2

Table 7. GFLOP/s and the number of processed cells per second on MILK RISC-V nodes.

Nodes	GFLOP/s	Processed cells per second
1	2.65	599 061
2	4.55	1 028 090

to two nodes. Currently, only two MILK RISC-V nodes are available to us and we hope to do larger runs on future clusters.

Additionally, we have performance results running on NVIDIA Grace Hopper on an early access node. However, only node-level results are available, and distributed runs on LANL’s Venado using the finalized hardware and software stack are planned. Node-level scaling on Intel Xeon Max CPUs and Intel Max GPUs are available and runs on Aurora are targeted.

Conclusion and outlook

The simulation of large time-dependent, three-dimensional, multi-scale, multi-model physical systems based on dynamic and adaptive meshes poses grand challenges to the fastest supercomputers. In this work, we have pioneered the simulation of stellar mergers on massively parallel systems with heterogeneous hardware and made significant contributions to both the application domain and high-performance computing methodology.

We have created Octo-Tiger, an astrophysical code that can simulate astrophysical phenomena such as the merger of binary star systems. Exemplary for many applications, we need to employ dynamic adaptive mesh refinement to efficiently bridge across multiple physical scales. In contrast to alternative approaches based on smooth particle hydrodynamics, a mesh-based finite volume scheme is used to allow us to conserve important physical quantities such as momentum and energy up to machine precision. Additionally, it enables convergence studies that are critical to a deeper understanding of the underlying phenomena. We realize, for the first time, highly resolved simulations with up to 257 million dynamically adaptive cells, which was previously infeasible. With this work, we have prepared the groundwork for extreme-scale simulations to gain a deeper understanding of the evolution of our universe.

Dynamic and adaptive meshes covering multi-physics (such as hydrodynamics and gravity) are, however, a clear mismatch with modern massively distributed and heterogeneous high-performance computers. Dynamic load balancing and code portability are but two of the grand challenges.

We solve load balancing by employing asynchronous many task parallelism via the graph-based scheduling of HPX, Kokkos kernels, and lightweight, non-blocking communication resources. We have pioneered the

integration of Kokkos, HPX, and SIMD-types to achieve code portability across a spectrum of heterogeneous supercomputers including Perlmutter, Frontier, and Fugaku, encompassing all three major GPU architectures as well as x86, ARM, and RISC-V CPUs.

To care for the communication requirements of adaptive algorithms, asynchronous programming models, and heterogeneous systems, which are usually heavily multi-threaded with finer-grained, point-to-point, concurrent communication, we have contributed a seamless switch of the underlying communication libraries at run time. Besides classical MPI, we have shown the benefits of employing an alternative communication library (LCI) as the communication backend to significantly improve communication efficiency.

In our largest runs with 257 million dynamically adaptive cells, we have achieved 51.37% parallel efficiency on Frontier on 32,768 cores and 2048 MI250X GPUs with about 17% HPCG peak performance; 64.47% on 294,912 cores on supercomputer Fugaku (5% HPCG peak); and 47.59% for a full system run on Perlmutter on 4,110,080 cores, and 6880 A100 GPUs using both the available 40 GB A100 nodes with the 80 GB A100 nodes simultaneously (26% HPCG peak).

Our contributions have also brought astrophysical simulations into a unique position to gain code and performance portability for novel and future systems. On a node level, we have already shown code portability to the new Intel GPUs, to Raspberry PIs, and to RISC-V CPUs. These efforts are pivotal in laying the groundwork for its use on tomorrow’s upcoming ARM64 and RISC-V HPC systems such as the European RISC-V flagship supercomputer announced for 2026.

These computational and algorithmic achievements are not specific to our guiding astrophysical application. The insights and significant improvements gained can be directly transferred to other mesh-based multi-physics codes with dynamic adaptivity, thus promising a high impact on future code developments. However, while the scalability and portability results with Octo-Tiger look extremely promising, there is still much to improve within the application itself, as is shown by the achieved FLOP/s. One major downside of the current version of Octo-Tiger, is that it is a GPU-accelerated application, not yet a GPU-resident one. While its computational hotspots within the hydro and gravity solvers have been ported to Kokkos (and thus to GPUs), there are many parts of its code that still run on CPU. This means that the GPU results constantly need to be transferred to the CPU to be processed there by other parts of code. Worse, as we had to refactor some utilized data-structures to make Octo-Tiger more suitable for GPUs (moving from Array-of-Structs to Struct-of-Arrays, for example, see (Pfander et al., 2018)), the interface between these older and newer parts sometimes includes data-structure conversions.

This puts a lot of stress on the main memory bandwidth, turning it into a bottleneck. This is mostly a result of refactoring and porting Octo-Tiger piece by piece. Despite this bottleneck, we still achieve major speedups when using GPUs over CPUs (Daiß et al., 2021; Daiß et al., 2023; Daiß et al., 2022b; Diehl et al., 2021). However, the aforementioned legacy CPU code parts prevent us from realizing Octo-Tiger's full potential on GPU machines just yet. Thus, our next steps regarding Octo-Tiger include finally removing these older parts of Octo-Tiger, focusing on the ones that include the data-structure conversion. Our goal is to eventually turn Octo-Tiger into a fully GPU-resident application, with all its main data-structures per sub-grid being located on the GPU.

Acknowledgements

This research used resources of the National Energy Research Scientific Computing Center (NERSC), a U.S. Department of Energy Office of Science User Facility located at Lawrence Berkeley National Laboratory, operated under Contract No. DE-AC02-05CH11231 using NERSC award DDR-ERCAP0028472. This research used resources of the Oak Ridge Leadership Computing Facility at the Oak Ridge National Laboratory, which is supported by the Office of Science of the U.S. Department of Energy under Contract No. DE-AC05-00OR22725. This research used computational resources of the supercomputer Fugaku provided by RIKEN Center for Computational Science. This work was supported by the U.S. Department of Energy through the Los Alamos National Laboratory (LANL). LANL is operated by Triad National Security, LLC, for the National Nuclear Security Administration of U.S. Department of Energy (Contract No. 89233218CNA000001). We also thank the LANL Advanced Simulation & Computing Program and CCS-7 Darwin cluster for computational resources. The authors would like to thank Stony Brook Research Computing and Cyberinfrastructure, and the Institute for Advanced Computational Science at Stony Brook University for access to the innovative high-performance Ookami computing system, which was made possible by a \$5M National Science Foundation grant (#1927880). The support we received from the Center of Computation and Technology at Louisiana State University was invaluable. The authors also acknowledge the technical support we received from NVIDIA (Scot Halverson) in the early stages of the project. Assigned: LA-UR-24-23457 (Rev. 3).

ORCID iDs

Patrick Diehl  <https://orcid.org/0000-0003-3922-8419>
 Christoph Junghans  <https://orcid.org/0000-0003-0925-1458>
 Alexander Straub  <https://orcid.org/0000-0002-6749-9710>

Funding

The authors disclosed receipt of the following financial support for the research, authorship, and/or publication of this article: This work was supported by the National Science Foundation, 1927880,

National Energy Research Scientific Computing Center; DDR-ERCAP0028472, Lawrence Berkeley National Laboratory DE-AC02-05CH11231, Office of Science of the U.S. Department of Energy DE-AC05-00OR22725, U.S. Department of Energy through the Los Alamos National Laboratory, National Nuclear Security Administration of U.S. Department of Energy 89233218CNA000001.

Declaration of conflicting interests

The authors declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Notes

1. <https://hpx-docs.stellar-group.org/latest/html/index.html>.
2. Merged into GCC in <https://github.com/gcc-mirror/gcc/commit/9ac3119fec81fb64d11dee8f853145f937389366>.
3. <https://github.com/STELLAR-GROUP/OperationBell24>.

References

- Ajima Y, Kawashima T, Okamoto T, et al. (2018) The tofu interconnect D. In: *2018 IEEE International Conference on Cluster Computing (CLUSTER)*, 646–654.
- Almgren A, Sazo M, Bell J, et al. (2020) CASTRO: a massively parallel compressible astrophysics simulation code. *Journal of Open Source Software* 5(54): 2513.
- Bangerth W, Hartmann R and Kanschat G (2007) Deal. II—A general-purpose object-oriented finite element library. *ACM Transactions on Mathematical Software* 33(4): 24.
- Bauer M, Treichler S, Slaughter E, et al. (2012) Legion: expressing locality and Independence with logical regions. In: *SC'12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–11.
- Beckingsale DA, Burmark J, Hornung R, et al. (2019) RAJA: portable performance for large-scale scientific applications. In: *2019 IEEE/ACM International Workshop on Performance, Portability and Productivity in Hpc (P3Hpc)*. IEEE, 71–81.
- Bonachea D and Hargrove PH (2018) Gasnet-ex: a high-performance, portable communication library for exascale. In: *International Workshop on Languages and Compilers for Parallel Computing*. Springer, 138–158.
- Bosilca G, Bouteiller A, Danalis A, et al. (2013) Parsec: exploiting heterogeneity to enhance scalability. *Computing in Science & Engineering* 15(6): 36–45.
- Burstedde C, Wilcox LC and Ghattas O (2011) p4est: scalable algorithms for parallel adaptive mesh refinement on forests of octrees. *SIAM Journal on Scientific Computing* 33(3): 1103–1133.
- Chamberlain BL (2011) *Chapel (Cray Inc. HPCS Language)*, Chapter C. Springer US, 249–256.
- Chatzopoulos E, Frank J, Marcello DC, et al. (2020) Is Betelgeuse the outcome of a past merger? *The Astrophysical Journal* 896(1): 50.

- Colella P, Graves D, Johnson J, et al. (2012) *Chombo Software Package for AMR Applications Design Document*. Lawrence Berkeley National Laboratory.
- Core Developers K (2024) Kokkos documentation. Last accessed. <https://kokkos.org/kokkos-core-wiki/>
- Daiß G, Amini P, Biddiscombe J, et al. (2019) From Piz Daint to the stars: simulation of stellar mergers using high-level abstractions. In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 1–37.
- Daiß G, Simberg M, Reverdell A, et al. (2021) Beyond Fork-join: integration of performance portable kokkos kernels with HPX. In: *2021 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, 377–386.
- Daiß G, Singanaboina SY, Diehl P, et al. (2022a) From merging frameworks to merging stars: experiences using hpx, kokkos and simd types. In: *2022 IEEE/ACM 7th International Workshop on Extreme Scale Programming Models and Middleware (ESPM2)*, 10–19.
- Daiß G, Diehl P, Marcello D, et al. (2022b) From task-based GPU work aggregation to stellar mergers: turning fine-grained CPU tasks into portable GPU kernels. In: *2022 IEEE/ACM International Workshop on Performance, Portability and Productivity in HPC (P3HPC)*, 89–99.
- Daiß G, Diehl P, Kaiser H, et al. (2023) Stellar mergers with HPX-Kokkos and SYCL: methods of using an asynchronous many-task runtime system with SYCL. In: *Proceedings of the 2023 International Workshop on OpenCL, IWOCL '23*. Association for Computing Machinery, 1–12.
- Diehl P, Daiß G, Marcello D, et al. (2021) Octo-Tiger's new hydro module and performance using HPX + CUDA on ORNL's summit. In: *2021 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 204–214.
- Diehl P, Brandt S and Kaiser H (2024a) *Parallel C++: Efficient and Scalable High-Performance Parallel Programming Using HPX*. Springer Nature.
- Diehl P, Syskakis P, Daiß G, et al. (2024b) Preparing for HPC on RISC-V: examining vectorization and distributed performance of an astrophysics application with HPX and Kokkos. URL: <https://arxiv.org/abs/2407.00026>
- Diehl P, Daiß G, Huck K, et al. (2024c) Simulating stellar merger using HPX/Kokkos on A64FX on supercomputer Fugaku. *The Journal of Supercomputing* 80: 16947–16978.
- Edwards HC, Trott CR and Sunderland D (2014) Kokkos: enabling manycore performance portability through polymorphic memory access patterns. *Journal of Parallel and Distributed Computing* 74(12): 3202–3216, Domain-Specific Languages and High-Level Frameworks for High-Performance Computing.
- Fryxell B, Olson K, Ricker P, et al. (2000) FLASH: an adaptive mesh hydrodynamics code for Modeling Astrophysical thermonuclear flashes. *The Astrophysical Journal - Supplement Series* 131(1): 273–334.
- Germain JDS, McCorquodale J, Parker SG, et al. (2000) Uintah: a massively parallel problem solving environment. In: *Proceedings the Ninth International Symposium on High-Performance Distributed Computing*. IEEE, 33–41.
- Heroux MA, Dongarra J and Luszczek P, (2013) *HPCG Benchmark Technical Specification*. Sandia National Lab.(SNL-NM). Technical report.
- Kaiser H, Simberg M, Diehl P, et al. (2023) STELLAR-GROUP/hpx: HPX V1.9.1: the C++ standards library for parallelism and concurrency.
- Kaiser H, Heller T, Adelstein-Lelbach B, et al. (2014) HPX: a task based programming model in a global address space. In: *Proceedings of the 8th International Conference on Partitioned Global Address Space Programming Models, PGAS '14*. Association for Computing Machinery, 1–11.
- Kaiser H, Diehl P, Lemoine A, et al. (2020) HPX - the C++ standard library for parallelism and concurrency. *Journal of Open Source Software* 5(53): 2352.
- Kale LV and Krishnan S (1993) Charm++: a portable concurrent object oriented system based on C++. In: *Proceedings of the Eighth Annual Conference on Object-Oriented Programming Systems, Languages, and Applications*, 91–108.
- Langer A, Lifflander J, Miller P, et al. (2012) Scalable algorithms for distributed-memory adaptive mesh refinement. In: *Proceedings of the 2012 IEEE 24th International Symposium on Computer Architecture and High Performance Computing, SBAC-PAD '12*. IEEE Computer Society, 100–107.
- Lauer A, Chatzopoulos E, Clayton GC, et al. (2019) Evolving R corone borealis stars with MESA. *Monthly Notices of the Royal Astronomical Society* 488(1): 438–450.
- MacNeice P, Olson KM, Mobarry C, et al. (2000) PARAMESH: a parallel adaptive mesh refinement community toolkit. *Computer Physics Communications* 126(3): 330–354.
- Marcello DC, Shiber S, De Marco O, et al. (2021) Octo-tiger: a new, 3D hydrodynamic code for stellar mergers that uses HPX parallelisation. *Monthly Notices of the Royal Astronomical Society* 504(4): 5345–5382.
- O'shea BW, Bryan G, Bordner J, et al. (2005) Introducing Enzo, an AMR cosmology application. In: *Adaptive Mesh Refinement-Theory and Applications: Proceedings of the Chicago Workshop on Adaptive Mesh Refinement Methods*. Springer, 341–349.
- Pfander D, Daiß G, Marcello D, et al. (2018) Accelerating octo-tiger: stellar mergers on intel knights landing with hpx. In: *Proceedings of the International Workshop on OpenCL*, 1–8.
- Poole SW, Hernandez O, Kuehn JA, et al. (2011) *OpenSHMEM - Toward a Unified RMA Model, Chapter O*. Springer US, 1379–1391.
- Sato M, Ishikawa Y, Tomita H, et al. (2020) Co-Design for A64FX manycore processor and “Fugaku”. In: *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, 1–15.
- Shiber S, De Marco O, Motl PM, et al. (2024a) Hydrodynamic simulations of white dwarf-white dwarf mergers and the origin of r corone borealis stars. *Monthly Notices of the Royal Astronomical Society* 535(2): 1914–1943.

- Shiber S, Marco OD, Motl PM, et al. (2024b) Hydrodynamic simulations of wd-wd mergers and the origin of rcb stars.
- Snir M and Yan J (2023) LCI: a lightweight communication interface v1. URL. <https://github.com/uiuc-hpc/LC/blob/icpp23/doc/LCI.pdf>
- Staff JE, Menon A, Herwig F, et al. (2012) Do r coronae borealis stars form from double white dwarf mergers? *The Astrophysical Journal* 757(1): 76.
- Sunderland D, Peterson B, Schmidt J, et al. (2016) An overview of performance portability in the uintah runtime system through the use of kokkos. In: *2016 Second International Workshop on Extreme Scale Programming Models and Middleware (ESPM2)*. IEEE, 44–47.
- The C++ Standards Committee (2018) ISO international standard ISO/IEC TS 19570:2018, technical specification for C++ extensions for parallelism. Geneva, Switzerland: International Organization for Standardization (ISO), URL. <https://www.open-std.org/jtc1/sc22/wg21>
- Thoman P, Dichev K, Heller T, et al. (2018) A taxonomy of task-based parallel programming technologies for high-performance computing. *The Journal of Supercomputing* 74(4): 1422–1434.
- Trott CR, Lebrun-Grandie D, Arndt D, et al. (2022) Kokkos 3: programming model extensions for the exascale era. *IEEE Transactions on Parallel and Distributed Systems* 33(4): 805–817.
- Wissink AM, Hornung RD, Kohn SR, et al. (2001) Large Scale Parallel Structured AMR Calculations Using the SAMRAI Framework, SC '01: Proceedings of the 2001 ACM/IEEE Conference on Supercomputing, Denver, CO, USA, 2001, pp. 22–22, doi: [10.1145/582034.582040](https://doi.org/10.1145/582034.582040).
- Yan J, Kaiser H and Snir M, (2023) Design and analysis of the network software stack of an asynchronous many-task system – the LCI parcellport of HPX. In: *Proceedings of the SC '23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis, SC-W '23*. Association for Computing Machinery, 1151–1161.
- Zhang W, Myers A, Gott K, et al. (2021) AMReX: block-structured adaptive mesh refinement for multiphysics applications. *The International Journal of High Performance Computing Applications* 35(6): 508–526.

Author biographies

Gregor Daifß is a PhD student at the University of Stuttgart, specializing in high-performance computing. His research focuses on task-based runtime systems and how to enable efficient GPU usage with them for highly adaptive and distributed scientific applications. His current work is concentrated on the development of dynamic GPU kernel fusion with the task-based runtime system HPX and on better integrating HPX with SYCL and Kokkos. It also places importance on leveraging these developments in real-world scientific applications for better performance and

portability. This also includes refactoring these scientific applications and porting them to accelerators in the first place.

Dr. Patrick Diehl is a computational scientist at **Los Alamos National Laboratory (LANL)** specializing in **high-performance computing, computational mechanics**, and **numerical methods**. His work focuses on scalable algorithms and open-source software for large-scale simulations in science and engineering. He is the **co-author of *Discovering Modern C++: An Intensive Course for Scientists, Engineers, and Programmers*** and serves as an **editor for the Journal of Open Source Software (JOSS)**. Patrick is an advocate for open science, reproducibility, and sustainable software development in computational research.

Jiakun Yan is a sixth-year Ph.D. student in the Siebel School of Computing and Data Science at the University of Illinois Urbana-Champaign, advised by Prof. Marc Snir. His research focuses on designing high-performance communication libraries for highly dynamic and irregular programming systems and applications. He is the lead developer of the Lightweight Communication Interface (LCI) and has also contributed to MPICH, HPX, Legion, and Charm++.

John K. Holmen is an HPC Engineer in the System Acceptance and User Environment group at the National Center for Computational Sciences at Oak Ridge National Laboratory. He earned his B.S. in Computer Engineering and Electrical Engineering from Kettering University, his M.S.E. in Computer Engineering from Kettering University, and his Ph.D. in Scientific Computing from the University of Utah. His research interests lie in High-Performance Computing and center around a deep-rooted passion for understanding how to make effective use of arbitrary hardware and software stacks in computer systems.

Rahul Kumar Gayatri is an HPC Programming Model Architect in the Programming Environments and Models (PEM) group at NERSC. His work focuses on developing programming models and optimizing application performance on GPU architectures. He is also involved in the development and maintenance of user-facing software on NERSC machines.

Christoph Junghans leads the applied computer science group at Los Alamos National Laboratory, where his research focuses on multiscale modeling, computational co-design, and various other interesting topics. He holds a MSc from Leipzig University and PhD from the Max-Planck-Institute for Polymer Research, both in physics.

Alexander Straub received his M.Sc. degree in Computer Science from the University of Stuttgart, Germany, where he is currently pursuing his Ph.D. at the Visualization Research Center (VISUS) in the group of Prof. Daniel Weiskopf. His research focuses on scientific visualization, particularly on

the visual analysis and exploration of multiphase flow phenomena and interface dynamics in porous media. He is affiliated with the Collaborative Research Centre **SFB 1313: Interface-Driven Multi-Field Processes in Porous Media – Flow, Transport and Deformation**, where he contributes to *Project D01: Visualization of Multi-Field Processes in Porous Media*. In addition, he is a member of the **Research Training Group GRK 2160 (DROPIT)**, working on visualization methods for droplet-based multiphase flow simulations. His recent work includes developing visualization techniques for deformation and interface behavior in multiphase systems, as well as collaborative studies bridging visualization, simulation, and fluid dynamics.

Jeff Hammond is a Principal Engineer at NVIDIA in the data center software organization, focused on GPU communications. Previously, he worked for Intel and Argonne National Laboratory. Jeff has extensive experience with the design and use of parallel programming models and scientific applications. His most notable achievements include the MPI-5 Application Binary Interface standard, development of the MPI-3 one-sided communication software ecosystem, and contributions to the NWChem quantum chemistry project. Jeff received a PhD in Chemistry from the University of Chicago in 2009.

Dominic C. Marcello is a Research Scientist with the Louisiana State University Center for Computation & Technology. His research centers on high-performance computational methods for astrophysics, with emphasis on adaptive-mesh hydrodynamics, self gravity, radiation transport, and asynchronous many-task runtimes. He is the lead developer of Octo-Tiger, an open-source, 3D code for simulating mass transfer and mergers in interacting binaries, used for studies of double white dwarf systems and related transients. He received his B.S. (2000) and Ph.D. (2011) in physics from Louisiana State University.

Miwako Tsuji is a professor of Center for Computational Sciences, University of Tsukuba, and a unit leader at the RIKEN Center for Computational Science. She received master and PhD degrees from the Information Science and

Technology department at Hokkaido University. From 2007 to 2013, she worked in multiple roles at the University of Hokkaido, University of Tokyo, University of Tsukuba, and Universite de Versailles Saint-Quentin-en-Yvelines. At RIKEN, she is a member of the Flagship 2020 project, which conducted the design and development of the supercomputer Fugaku. Her current research interests are programming and performance models for large-scale high-performance computing. She is a coauthor of the ACM Gordon Bell Prize in 2011. She is one of the Associate Directors of the Arm HPC User Group organization.

Dirk Pflüger is Professor for Scientific Computing at the Institute for Parallel and Distributed Systems and fellow of the SimTech Cluster of Excellence at the University of Stuttgart. He received an MSc from the University of Stuttgart, a Master's degree from the University of Sydney, and a PhD from the Technical University of Munich. His research interests include high-dimensional problems and simulations; sparse grids; scientific ML; data-driven problems and their efficient implementation; and performance portability, hardware-aware hybrid parallelizations, and high-performance computing.

Hartmut Kaiser is a professor at Louisiana State University. He is probably best known for his involvement in open-source software projects, such as being the author of several C++ libraries he has contributed to Boost that are in use by thousands of developers worldwide. In his current research he focusses on leading the STE||AR Group at LSU's Center for Computation and Technology working on the practical design and implementation of future execution models and programming methods using the HPX runtime system - a modern C++ library for concurrency and parallelism. His research interests are focused on the complex interaction of compiler technologies, runtime systems, active libraries, and modern system's architectures. His goal is to enable the creation of a new generation of scientific applications in powerful, though complex environments, such as high-performance computing, distributed and cloud computing, spatial information systems, and compiler technologies.